AI Guess Service: Evaluating LLM Response Times and Inference Quality in Distributed Environments

Kelly Franck Ishaka
Research Internship - Distributed Systems & AI

Abstract

This paper presents a simple but practical system for integrating a large language model into a web service in a distributed setting. The system receives user descriptions and uses an LLM (DeepSeek-R1 via Ollama) to generate a single-word guess. The focus of this work is not only functionality but also understanding real-world constraints such as latency, observability, and deployment. Experimental results show that while reasoning-focused models provide accurate predictions, inference latency (10–20 seconds) remains a key limitation. The system is containerized using Docker to ensure portability across environments.

Keywords

Distributed Systems, Large Language Models, Web Services, Performance Evaluation, Containerization

ACM Reference Format:
Kelly Franck Ishaka. . AI Guess Service: Evaluating LLM Response Times and Inference Quality in Distributed Environments. In *Proceedings of* . ACM, New York, NY, USA, 2 pages.

1 Introduction

Integrating large language models (LLMs) into real-world applications introduces several practical challenges, particularly in distributed systems. While LLMs are powerful in understanding and generating language, deploying them requires careful handling of latency, system design, and resource constraints.

This work presents a minimal yet functional system that connects a web service to a large language model using a distributed architecture. The goal is to explore how such systems behave in practice, focusing on performance, simplicity, and observability.

The main objectives of this work are:

- To design a simple end-to-end system connecting a web interface to an LLM
- To measure real inference latency in a practical setup
- To ensure the system is portable using containerization
- To reflect on design decisions and trade-offs

2 System Design

The system consists of three main components:

Frontend: A minimal HTML/JavaScript frontend that accepts user descriptions and displays model predictions. The interface emphasizes simplicity and human-readable feedback.

Backend: A FastAPI service that processes requests, sends them to the model, and returns results. It also tracks basic statistics such as response time and number of requests.

LLM Runtime: The DeepSeek-R1 model is executed using Ollama on a separate machine.

2.1 Architecture

`Client` → `FastAPI` → `SSH Tunnel` → `Ollama` → `Model` → `Response`

To enable communication with the remote model, SSH port forwarding is used. This allows the local API to interact with the remote LLM as if it were running locally.

2.2 Communication

The system uses synchronous HTTP requests between components. A timeout of 120 seconds is set to accommodate slow responses from the reasoning model.

3 Implementation

The system is implemented using the following technologies:

- FastAPI (web framework)
- Uvicorn (ASGI server)
- Requests (HTTP client)
- Pydantic (data validation)
- Ollama (LLM runtime)
- Docker (containerization)

The backend sends a request to the Ollama API and receives a response. Since the model sometimes includes reasoning text, a cleaning step is applied to extract only the final one-word answer.

4 Evaluation

4.1 Performance

During testing, the following observations were made:

- Inference latency: 10–20 seconds per request
- Total response time: 10.5–20.5 seconds
- Accuracy: high for simple descriptive prompts

4.2 Observations

It became clear that the main limitation was not the API or network, but the model's inference time. The reasoning capabilities of DeepSeek-R1 improve accuracy but significantly increase latency.

5 Design Decisions

Several decisions were made to keep the system simple and understandable:

- **Simplicity first:** No database or complex architecture was used initially
- **Remote model usage:** Offloading computation to a remote server avoids local hardware limitations

, ,
.

- **SSH tunneling:** Provides a simple and secure connection method
- **Output cleaning:** Ensures responses match the requirement (single word)
- **Basic observability:** Response time and request count are tracked

6 Limitations

This implementation is intentionally minimal and has several limitations:

- No persistent storage (statistics reset on restart)
- High latency due to large model size
- No asynchronous processing
- Limited evaluation dataset

7 Work Status and Future Improvements

Due to the one-week time constraint, the system focuses on core functionality. Additional improvements would include:

- Adding persistent storage (e.g., database)
- Implementing caching for repeated queries
- Using asynchronous processing
- Evaluating smaller models for lower latency
- Deploying with orchestration tools (e.g., Kubernetes)

These improvements would require approximately 3–5 additional days of development.

8 Conclusion

This work demonstrates a simple but effective way to integrate an LLM into a web service using a distributed approach. While the system performs well functionally, inference latency remains a key challenge. The project highlights the importance of balancing accuracy, performance, and system design in real-world AI applications.

Acknowledgment of AI Assistance

Parts of this work, including debugging support, structuring ideas, and refining documentation, were assisted by generative AI tools. All implementation, testing, and final decisions were performed by the author.

References

[1] Ollama. https://ollama.ai
[2] FastAPI. https://fastapi.tiangolo.com
[3] DeepSeek-R1 Technical Report. https://github.com/deepseek-ai/DeepSeek-R1
[4] Docker. https://www.docker.com
[5] Sam Newman. Building Microservices. O'Reilly Media, 2015.
[6] Hennessy, J., Patterson, D. Computer Architecture: A Quantitative Approach, 2019.